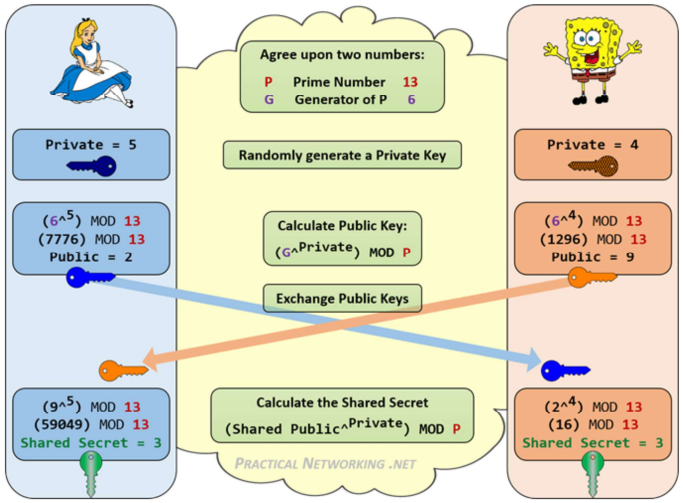
Diffie–Hellman Algorithm



© 2018 Al-Nafi. All Rights Reserved. 1

My own definition:

Diffie–Hellman is a key negotiation algorithm and does not provide for message confidentiality. It is used to enable two entities to exchange or negotiate a secret symmetric key that will be used subsequently for message encryption using symmetric key cryptography. The Diffie–Hellman algorithm can be extremely useful for applications such as PKI and others where the generation of symmetric session keys are required. It is often referred to as a session key negotiation algorithm. Diffie–Hellman is based on discrete logarithm hard math problem.

Diffie–Hellman can be summarized as follows: It is a key agreement protocol whereby two parties, without any prior arrangements, can agree upon a secret symmetric key that is known only to them. This secret key can then be used, for example, to encrypt further communications between the parties from that point on using symmetric key cryptography.
The Diffie–Hellman key agreement requires that both the sender and recipient of a message have their private and
public key pairs.

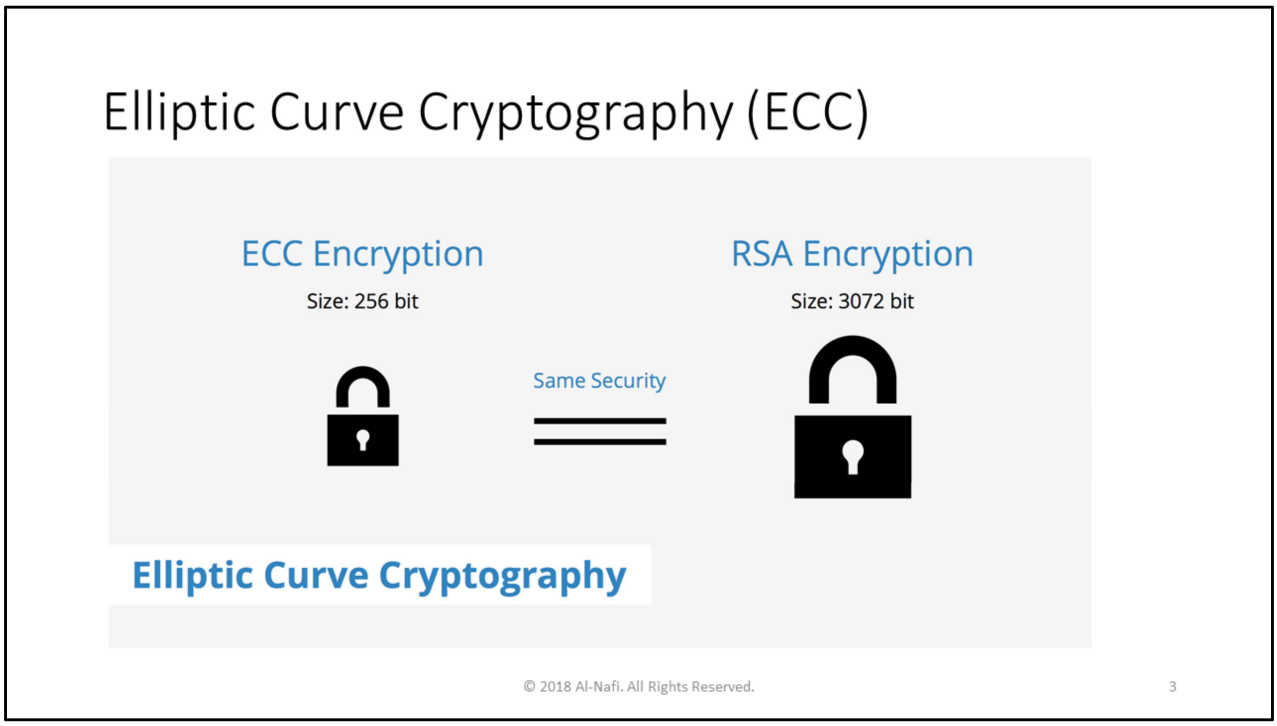
By combining one's private key and the other party's public key, both parties can compute the same shared secret number that ends up being the symmetric session key. A "session key" is a symmetric key that is used only for that particular session.

ElGamal Algorithm

The ElGamal cryptographic algorithm is based on the work of Diffie–Hellman, but it also includes the ability to provide message confidentiality and digital signature services, not just session key negotiation. Although not technically correct, some people refer to ElGamal as a combination of the Diffie–Hellman and RSA algorithms. The ElGamal algorithm is based on the same mathematical functions of discrete logs.

© 2018 Al-Nafi. All Rights Reserved.

The ElGamal cryptographic algorithm is based on the work of Diffie–Hellman, but it also includes the ability to provide message confidentiality and digital signature services, not just session key negotiation. Although not technically correct, some people refer to ElGamal as a combination of the Diffie–Hellman and RSA algorithms. The ElGamal algorithm is based on the same mathematical functions of discrete logs.



The elliptic curve algorithm has the highest strength per bit of key length of any of the asymmetric algorithms. The ability to use much shorter keys for elliptic curve cryptography (ECC) implementations provides savings on computational power, bandwidth, and storage. This makes ECC especially beneficial for implementations in smart cards, wireless, and other similar application areas where those elements may be lacking. Elliptic curve algorithms provide confidentiality, digital signatures, and message authentication services. The excitement of ECC is that elliptic curve group discrete log techniques have not seen significant improvement over the past number of years. This is obviously good news for elliptic methods because it allows us to use reduced key sizes to provide the same level of security as traditional public key cryptography methods.

Advantages and Disadvantages of Asymmetric Key Algorithms

Advantages

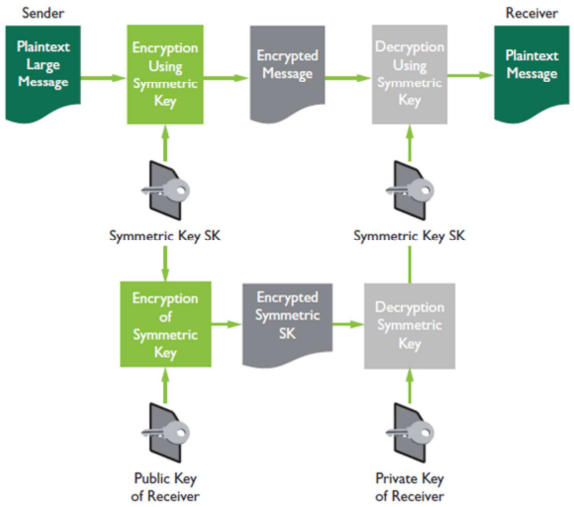
- Much stronger and complex than symmetric encryption.
- Only intended recipient can read the message.

Disadvantages

- More complex and requires larger infrastructure
- If private key is lost then its almost impossible to decrypt the message.

© 2018 Al-Nafi. All Rights Reserved.

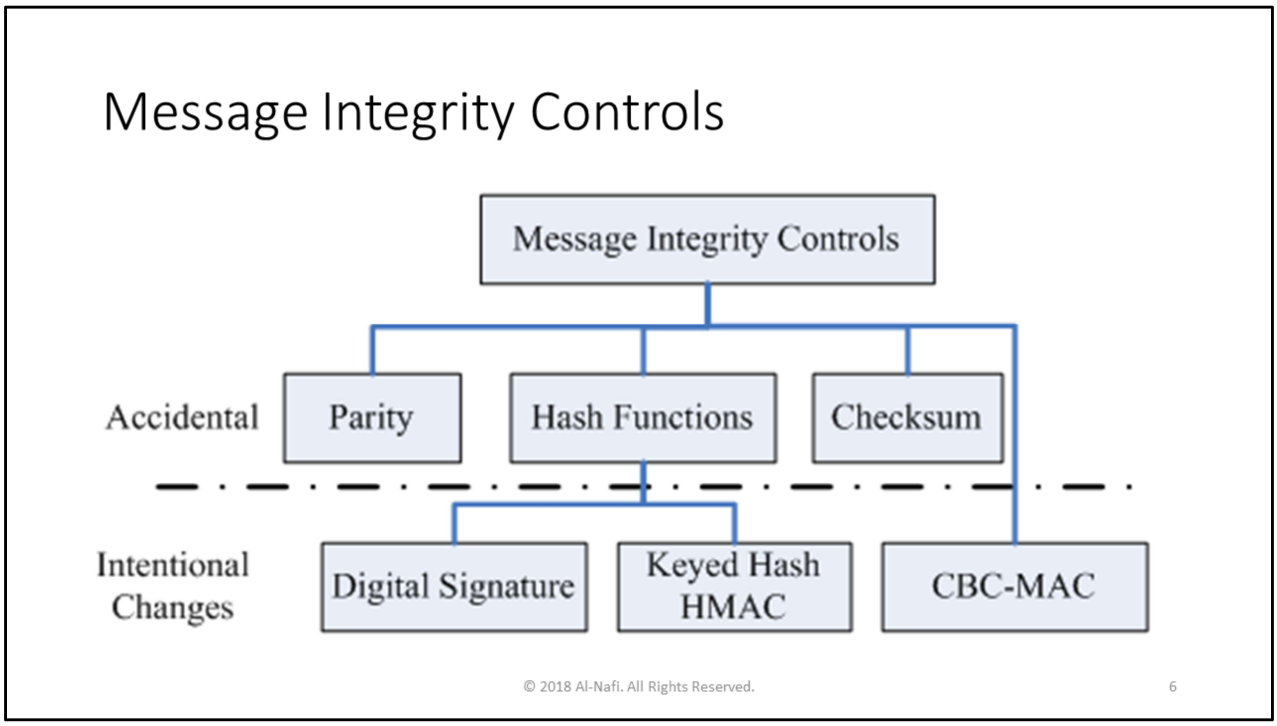
Hybrid Cryptography (The best of both worlds)



© 2018 Al-Nafi. All Rights Reserved.

5

Hybrid cryptography is where we use the advantages of both symmetric and asymmetric key cryptography. As you remember, symmetric is very fast but problematic in the way of key distribution. Asymmetric, on the other hand, is very slow but solves the problem of key distribution. Why not use both for what they are each good at? This is referred to as a hybrid cryptography system. A hybrid system operates as shown. The message itself is encrypted with a symmetric key, SK, and is sent to the recipient. To allow the recipient to have the symmetric key required for decryption, the symmetric key is encrypted with the public key of the recipient and sent to the recipient. The recipient then decrypts the symmetric key with their private key that no one else has. This provides the symmetric key to the recipient only. The symmetric key can then be used to decrypt the message.



An important part of electronic commerce and computerized transactions today is the assurance that a transmitted message or data has not been modified, is indeed from the person that the sender claims to be, and that the message was received by the correct party. This is accomplished through cryptographic functions that perform in several manners, depending on the business needs and level of trust between the parties and systems.

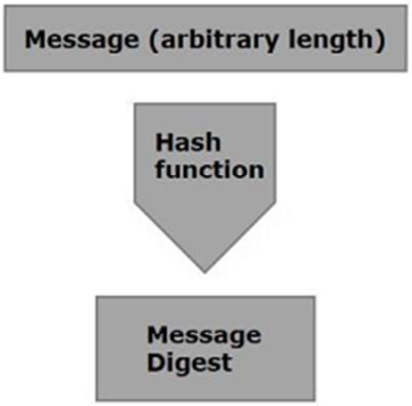
The point is this, when receiving messages over untrusted networks such as the internet, it is very important to ensure the integrity of the message. Integrity means receiving exactly what was sent, without modification. The principle of integrity assures that nothing changed without detection. In cryptography, this principle can be referred to as message authentication.

Message authentication can be achieved using Message Digest security features. Message digests come in two flavors: keyed and non-keyed.

Non-keyed message digests are made without a secret key and are called Message Integrity Codes (MICs). Most asymmetric key digital signature schemes use non-

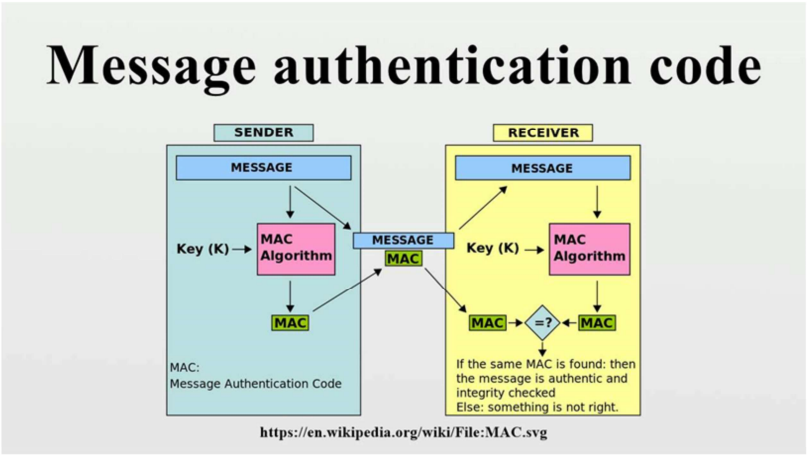
keyed message digests. Keyed message digests, known as Message Authentication Codes (MACs), combine a message digest and a secret key. MACs require the sender and the receiver to share a secret key ahead of time to be able to address integrity properly. It is important to realize that the word “keyed” does not mean that the message digest is signed (private key encrypted), instead, it means that the digest is encrypted with a secret symmetric key.

Message Digest



A message digest is a small representation of a larger message produced by a hashing algorithm. A message digest is used to ensure the integrity of information and does not address confidentiality of the message.

Message Authentication Code (MAC)

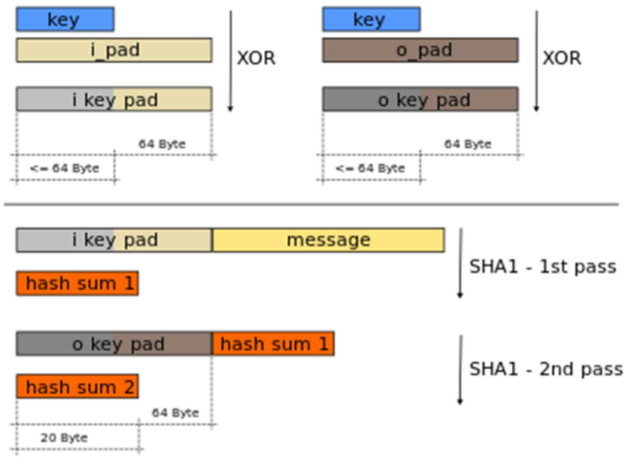


© 2018 Al-Nafi. All Rights Reserved.

A MAC (also known as a cryptographic checksum) is a small block of data that is generated using a secret key and then appended to the message. When the message is received, the recipient can generate their own MAC using the secret key, and thereby know that the message has not changed either accidentally or intentionally in transit. It is important to remember that this assurance is only as strong as the trust the two parties have that no one else has access to the secret symmetric key. A MAC is a small representation of a message and needs to have the following characteristics:

- A MAC is much smaller than the message generating it.
- Given a MAC, it is impractical to compute the message that generated it.
- Given a MAC and the message that generated it, it is impractical to find another message generating the same MAC.

Hash Message Authentication Code (HMAC)

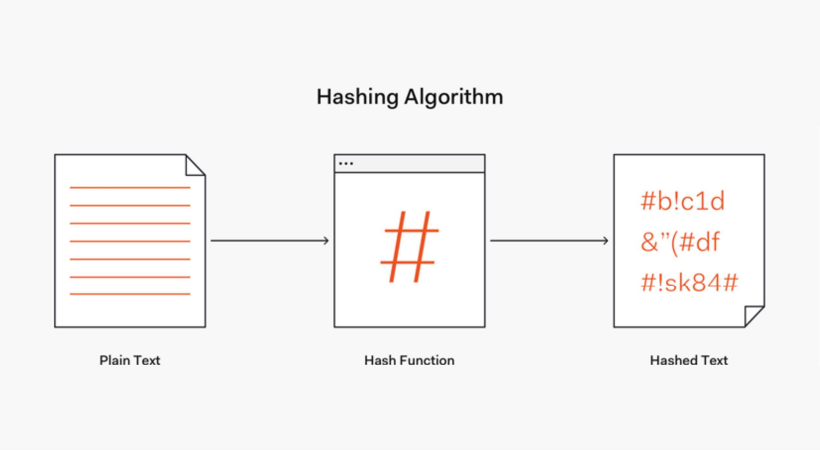


© 2018 Al-Nafi. All Rights Reserved.

Hashed MAC'ing implements a freely available hash algorithm (such as SHA-1 or MD5) as a component within the MAC implementation. This allows ease of the replacement of the hashing module if a new hash function ever becomes necessary.

The use of proven cryptographic hash algorithms also provides assurance of the security of HMAC implementations. HMACs work by adding a secret key value to the hash input function along with the source message. The HMAC operation provides cryptographic strength similar to a hashing algorithm, except that it now has the additional protection of a secret key and still operates nearly as rapidly as a standard hash operation.

Hashing



Hashing is defined as using a hashing algorithm to produce a message digest that can be used to address integrity. The hash function accepts an input message of any length and generates, through a one-way operation, a fixed-length output called a message digest. The difference between what we discussed above is that a hashing algorithm generates the message digest but does not use a secret key. There are several ways to use message digests in communications, depending on the need for the confidentiality of the message, the authentication of the source, the speed of processing, and the choice of encryption algorithms. The requirements for a hash function are that they must provide some assurance that the message has not changed without detection and that it would be impractical to find any two messages with the same message digest value.

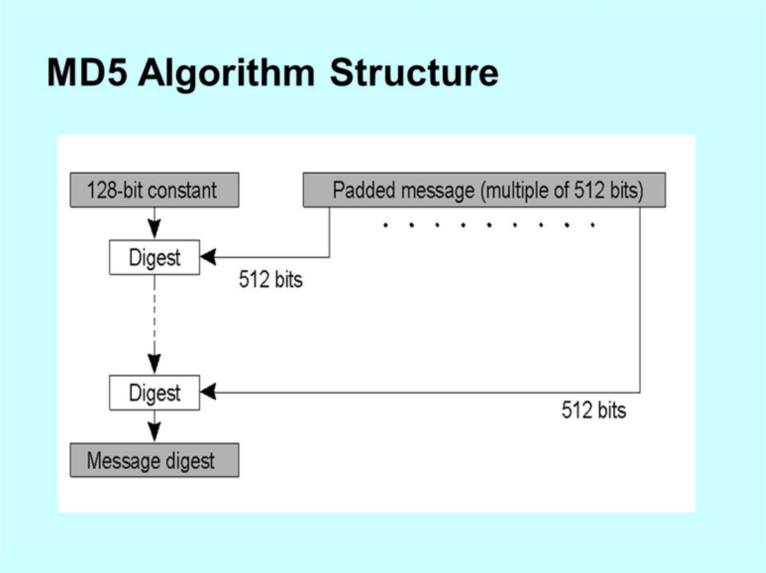
Examples of very popular hashing algorithms are SHA-1 and MD5.

Five Key Properties of a Hash Function

1. Uniformly distributed: The hash output value should not be predictable.
2. Collision resistant: Difficult to find a second input value that would hash to the

same value as another input, and difficult to find any two inputs that hash to the same value.
3. Difficult to invert: Should be one way, should not be able to derive the original message by reversing the hash.
4. Computed on the entire message: The hash algorithm should use the entire message to produce the digest.
5. Deterministic: Given an input *x*, it must always generate the same hash value, *y*.

MD5 (Message Digest 5) Message Digest Algorithm



© 2018 Al-Nafi. All Rights Reserved.

Reference https://searchsecurity.techtarget.com/definition/MD5

The MD5 hashing algorithm is a one-way cryptographic function that accepts a message of any length as input and returns as output a fixed-length digest value to be used for authenticating the original message.

The MD5 hash function was originally designed for use as a secure cryptographic hash algorithm for authenticating digital signatures. MD5 has been deprecated for uses other than as a non-cryptographic checksum to verify data integrity and detect unintentional data corruption.

Although originally designed as a cryptographic message authentication code algorithm for use on the internet, MD5 hashing is no longer considered reliable for use as a cryptographic checksum because researchers have demonstrated techniques capable of easily generating MD5 collisions on

commercial off-the-shelf computers.

Ronald Rivest, founder of RSA Data Security and institute professor at MIT, designed MD5 as an improvement to a prior message digest algorithm, MD4. Describing it in Internet Engineering Task Force RFC 1321, "The MD5 Message-Digest Algorithm," he wrote:

The algorithm takes as input a message of arbitrary length and produces as output a 128-bit 'fingerprint' or 'message digest' of the input. It is conjectured that it is computationally infeasible to produce two messages having the same message digest, or to produce any message having a given pre-specified target message digest. The MD5 algorithm is intended for digital signature applications, where a large file must be 'compressed' in a secure manner before being encrypted with a private (secret) key under a public-key cryptosystem such as RSA.

The IETF suggests MD5 hashing can still be used for integrity protection, noting "Where the MD5 checksum is used inline with the protocol solely to protect against errors, an MD5 checksum is still an acceptable use." However, it added that "any application and protocol that employs MD5 for any purpose needs to clearly state the expected security services from their use of MD5."

Message digest algorithm characteristics

Message digests, also known as hash functions, are one-way functions; they accept a message of any size as input, and produce as output a fixed-length message digest.

MD5 is the third message digest algorithm created by Rivest. All three (the others are MD2 and MD4) have similar structures, but MD2 was optimized for 8-bit machines, in comparison with the two later formulas, which are optimized for 32-bit machines. The MD5 algorithm is an extension of MD4, which the critical review found to be fast, but possibly not absolutely secure. In comparison, MD5 is not quite as fast as the MD4 algorithm, but offered much more assurance of data security.

How MD5 works

The MD5 message digest hashing algorithm processes data in 512-bit blocks, broken down into 16 words composed of 32 bits each. The output from MD5 is a 128-bit message digest value.

Computation of the MD5 digest value is performed in separate stages that process each 512-bit block of data along with the value computed in the preceding stage. The first stage begins with the message digest values initialized using consecutive hexadecimal numerical values. Each stage includes four message digest

passes which manipulate values in the current data block and values processed from the previous block. The final value computed from the last block becomes the MD5 digest for that block.

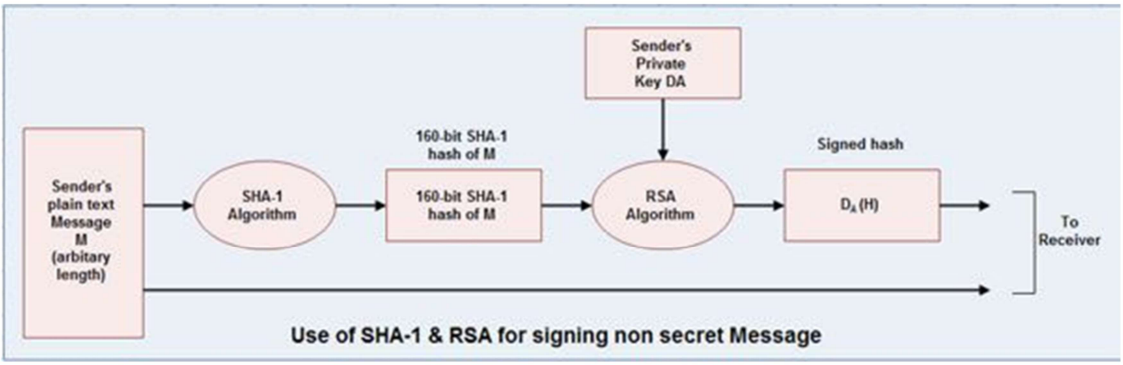
MD5 security

The goal of any message digest function is to produce digests that appear to be random. To be considered cryptographically secure, the hash function should meet two requirements: first, that it is impossible for an attacker to generate a message matching a specific hash value; and second, that it is impossible for an attacker to create two messages that produce the same hash value.

MD5 hashes are no longer considered cryptographically secure, and they should not be used for cryptographic authentication.

In 2011, the IETF published RFC 6151, "Updated Security Considerations for the MD5 Message-Digest and the HMAC-MD5 Algorithms," which cited a number of recent attacks against MD5 hashes, especially one that generated hash collisions in a minute or less on a standard notebook and another that could generate a collision in as little as 10 seconds on a 2.66 GHz Pentium 4 system. As a result, the IETF suggested that new protocol designs should not use MD5 at all, and that the recent research attacks against the algorithm "have provided sufficient reason to eliminate MD5 usage in applications where collision resistance is required such as digital signatures."

Secure Hash Algorithm (SHA) and SHA-1



© 2018 Al-Nafi. All Rights Reserved.

The original SHA was developed by NIST in the United States in 1993 and was issued as Federal Information Processing Standard (FIPS) 180. A revised version (FIPS 180-1) was issued in 1995 as SHA-1 (RFC 3174) with some improvements. SHA was based on the previous MD4 algorithm, whereas SHA-1 follows the logic of the MD5 hashing algorithm described above. SHA-1 operates on 512-bit blocks. The output hash, or message digest, is 160 bits in length. The processing includes four rounds of operations of 20 steps each. As in MD5, recently, there have been several attacks described against the SHA-1 algorithm to try and find collisions, despite it being considered to be considerably stronger than MD5. NIST has issued FIPS 180-4 that recognizes SHA-1, SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, and SHA 512/256 as a part of the Secure Hash Standard. The output lengths of the digests of these vary from 160 to 512 bits, typically identified by the number written after the SHA letters.

Hash of Variable Length (HAVAL)

HAVAL is a hashing algorithm with a variable length output message digest. It combines a variable length output with a variable number of rounds of operations on 1,024-bit input blocks.

© 2018 Al-Nafi. All Rights Reserved.

HAVAL is a hashing algorithm with a variable length output message digest. It combines a variable length output with a variable number of rounds of operations on 1,024-bit input blocks. The output message digest may be 128, 160, 192, 224, or 256 bits, and the number of rounds may vary from three to five. That gives 15 possible combinations of operations. HAVAL's claim to fame is it can operate 60 percent faster than MD5 when only three rounds are used and is just as fast as MD5 when it does five rounds of operation.

RIPEMD-160 (RACE Integrity Primitives Evaluation Message Digest)

The original algorithm (RIPEMD-128) has the same vulnerabilities as MD4 and MD5 and led to the improved RIPEMD-160 version. The output for RIPEMD-160 is 160 bits, and it operates similarly to MD5 on 512-bit blocks. It does twice the processing of SHA-1, performing five paired rounds of 16 steps each for 160 operations.

© 2018 Al-Nafi. All Rights Reserved.

The original algorithm (RIPEMD-128) has the same vulnerabilities as MD4 and MD5 and led to the improved RIPEMD-160 version. The output for RIPEMD-160 is 160 bits, and it operates similarly to MD5 on 512-bit blocks. It does twice the processing of SHA-1, performing five paired rounds of 16 steps each for 160 operations. As with any other hashing
algorithm, the benefit of increasing the size of the message digest output is to provide better protection against collisions, where two different messages produce the same message digest value.